Codes

Python Code (Exported from IPYNB)

```
# %% [markdown]
# # **Part A**

# %%
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from arch import arch_model
from statsmodels.tsa.stattools import adfuller
import os

# %%
import warnings
warnings.filterwarnings("ignore")

# %% [markdown]
# ## *Downloading Nvidia Stock Data*

# %%
ticker = "NVDA"
data = pd.DataFrame(yf.download(ticker, period="5y", interval="1d"))

# %%
data.info()

# %%
data = data.droplevel(level=1, axis=1)
data.head()

# %% [markdown]
# ## *Preprocessing Steps*

# %%
# Check Log Returns
data['Log_Returns'] = np.log(data['Close']/data['Close'].shift(1))

# Drop missing values
data.dropna(inplace=True)

print(data[['Close', 'Log_Returns']].head())

# %%
data.info()

# %% [markdown]
```

```
# ## *Plot Close v/s Log Returns*

# %%
plt.figure(figsize=(14,6))

# Plot original close prices
plt.subplot(3, 1, 1)
plt.plot(data['Close'])
plt.title(f'{ticker} Close Price')
plt.ylabel('Price')

# Plot log returns
plt.subplot(3, 1, 2)
plt.plot(data['Log_Returns'], color='orange')
plt.title(f'{ticker} Log Returns')
plt.ylabel('Log Returns')
plt.xlabel('Date')

plt.tight_layout()
plt.show()

# %% [markdown]
# Interpretation: We can see that the returns have a somewhat constant mean and do
not have any seasonality. However, we can see that the volatility is not constant,
with periods of high volatility and periods of low volatility mixed within the
data. We can surmise based on this that this series is a great candidate for the
GARCH model. The following section will help us in confirming this interpretation.

# %% [markdown]
# We can also use the Augmented Dickey-Fuller test to check the stationarity of
the data. If the ADF statistic is significant (p-value < 0.05), then we can say
that the data is stationary.

# %%
adfuller_result = adfuller(data['Close'])
print(f"ADF Statistic: {adfuller_result[0]}")
print(f"p-value: {adfuller_result[1]}")

# %% [markdown]
# Since, the ADF statistic is not significant, we can conclude that the data is
non-stationary. Hence, we can assume that the data will not be a great candidate
for the ARCH(1) model as that assumes that the data has a constant mean and a non-
constant variance, we can also assume that the data will be a great candidate for
the GARCH(1, 1) model as that assumes non-constancy of the mean and the variance.

# %% [markdown]
# ## *ARCH & GARCH Effect*

# %%
returns = data['Log_Returns'] * 100

# %%
arch_func = arch_model(returns, vol='ARCH', p=1)
arch_results = arch_func.fit(update_freq=5)
```

```
print(arch_results.summary())

# %% [markdown]
# Using the summary of the fitting of the ARCH(1) model on our scaled returns, we
can see that the alpha[1] coefficient is not significant (p-value is 0.154). Thus,
we should check the GARCH model's summary.

# %%
garch_func = arch_model(returns, vol='GARCH', p=1, q=1)
garch_results = garch_func.fit(update_freq=5)
print(garch_results.summary())

# %% [markdown]
# Based on the summary of the fitting of the GARCH(1, 1) model on our scaled
returns, we can see that both alpha[1] and beta[1] are significant (p-values are
less than 0.05). Hence, we can confidently reject the null hypothesis.

# %%
conditional_vol = garch_results.conditional_volatility
forecast = garch_results.forecast(horizon=90)
forecasted_vol = np.sqrt(forecast.variance.iloc[-1])

# %%
# Create index for forecast days (business days)
forecast_index = pd.date_range(start=conditional_vol.index[-1] +
pd.Timedelta(days=1), periods=90, freq='B')

# Plot
plt.figure(figsize=(15,6))
plt.plot(conditional_vol.index[-90:], conditional_vol[-90:], label='Conditional
Volatility (In-Sample)', color='blue')
plt.plot(forecast_index, forecasted_vol, label='Forecast Volatility (Next 90
Days)', color='red')
plt.title('GARCH(1,1): Conditional and 90-Day Forecast Volatility')
plt.xlabel('Date')
plt.ylabel('Volatility (Standard Deviation of Returns)')
plt.legend()
plt.tight_layout()
plt.show()

# %% [markdown]
# # **Part B**

# %%
from statsmodels.tsa.api import VAR
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.vector_ar.vecm import coint_johansen, VECM

# %%
BASE = os.getcwd()

DATASETS_DIR = os.path.join(BASE, 'datasets')
DATASET_FILE = "CMO-Historical-Data-Monthly.xlsx"
DATASET_PATH = os.path.join(DATASETS_DIR, DATASET_FILE)
```

```
# %%
data = pd.read_excel(DATASET_PATH, sheet_name='Monthly Prices', skiprows=6)

# %%
data.head()

# %%
data.info()

# %%
data.columns = data.columns.str.strip()
data = data.rename(columns={'Unnamed: 0': 'Year_Month'})
data['Year_Month'] = pd.to_datetime(data['Year_Month'].apply(lambda x:
x.replace('M', '-')), format='%Y-%m')
data.set_index('Year_Month', inplace=True)

# %%
data.head()

# %% [markdown]
# I will be selecting three categories for the Multivariate analysis for
comparison, namely:
#
# 1. Energy
#    1. Crude Oil
#    2. Natural Gas
#    3. Coal
# 2. Metals
#    1. Gold
#    2. Silver
#    3. Copper
#    4. Aluminium
# 3. Agri/Food
#    1. Wheat
#    2. Sugar

# %%
energy_cols = [
    ## Crude Oil
    'CRUDE_BRENT',
    'CRUDE_WTI',
    'CRUDE_DUBAI',
    ## Natural Gas
    'NGAS_US',
    'NGAS_EUR',
    'NGAS_JP',
    ## Coal
    'COAL_AUS',
    'COAL_SAFRICA'
]

metals_cols = [
    ## Gold
```

```
    'GOLD',
    ## Silver
    'SILVER',
    ## Copper
    'COPPER',
    ## Aluminium
    'ALUMINUM'
]

food_cols = [
    ## Wheat
    'WHEAT_US_HRW',
    'WHEAT_US_SRW',
    ## Sugar
    'SUGAR_EU',
    'SUGAR_US',
    'SUGAR_WLD'
]

# %%
def prepare_category_df(df, cols):
    for col in cols:
        if data[col].dtype == 'object':
            data[col] = pd.to_numeric(data[col], errors='coerce')
    subset = df[cols].dropna()
    log_df = np.log(subset)
    log_diff_df = log_df.diff().dropna()
    return log_df, log_diff_df

# %%
def adf_test(diff_df):
    print("ADF Test:")
    for col in diff_df.columns:
        stat, pval, *_ = adfuller(diff_df[col])
        result = "Stationary" if pval < 0.05 else "Non-Stationary"
        print(f"{col}: ADF Statistic = {stat:.3f}, p = {pval:.4f} → {result}")

# %%
def run_var(diff_df):
    model = VAR(diff_df)
    selected_lags = model.select_order(maxlags=12)
    lag = selected_lags.aic
    print(selected_lags.summary())
    print(f"Using AIC-selected lag = {lag}")
    results = model.fit(lag)
    print(results.summary())
    irf = results.irf(10)
    irf.plot(orth=False)
    plt.show()
    return results, lag

# %%
def run_johansen(log_df, lag_diff):
    jres = coint_johansen(log_df, det_order=0, k_ar_diff=lag_diff)
```

```
    print("Johansen Trace Stats:")
    for i, stat in enumerate(jres.lr1):
        print(f"r = {i}: statistic = {stat:.2f}, crit(95%) = {jres.cvt[i,1]}")
    return jres

# %%
def fit_vecm(log_df, coint_rank, lag_diff):
    vec = VECM(log_df, k_ar_diff=lag_diff, coint_rank=coint_rank)
    vec_res = vec.fit()
    print(vec_res.summary())
    return vec_res

# %%
def do_analysis(category, cols):
    print(f"\n{'='*35}")
    print(f"Analyzing {category} Commodities")
    print(f"{'='*35}")

    log_df, log_diff_df = prepare_category_df(data, cols)

    print("\nADF (Stationarity) Test:")
    adf_test(log_diff_df)

    print("\nRunning VAR Model:")
    var_results, selected_lag = run_var(log_diff_df)

    print("\nJohansen Cointegration Test:")
    johansen = run_johansen(log_df, lag_diff=selected_lag)

    # Determine cointegration rank (where trace stat > 95% critical value)
    trace_stats = johansen.lr1
    crit_vals = johansen.cvt[:,1]
    coint_rank = sum(trace_stats > crit_vals)
    print(f"\nEstimated Cointegration Rank: {coint_rank}")

    if coint_rank > 0:
        print("\nFitting VECM Model:")
        vecm_results = fit_vecm(log_df, coint_rank, selected_lag)
    else:
        print("No cointegration detected; skipping VECM.")

# %% [markdown]
# ## **Energy**

# %%
# Crude Oil
crude_oil_cols = [
    'CRUDE_BRENT',
    'CRUDE_WTI',
    'CRUDE_DUBAI'
]
# Natural Gas
natural_gas_cols = [
    'NGAS_US',
```

```
    'NGAS_EUR',
    'NGAS_JP'
]
# Coal
coal_cols = [
    'COAL_AUS',
    'COAL_SAFRICA'
]

# %%
do_analysis("Crude Oil", crude_oil_cols)

# %%
do_analysis("Natural Gas", natural_gas_cols)

# %%
do_analysis("Coal", coal_cols)

# %% [markdown]
# ## **Metals**

# %%
do_analysis("Metals", metals_cols)

# %% [markdown]
# ## **Agri/Food**

# %%
wheat_cols = [
    'WHEAT_US_HRW',
    'WHEAT_US_SRW'
]

sugar_cols = [
    'SUGAR_EU',
    'SUGAR_US',
    'SUGAR_WLD'
]

# %%
do_analysis("Wheat", wheat_cols)

# %%
do_analysis("Sugar", sugar_cols)
```

R Code

```
# Set the base directory for the project
base_dir <- getwd()
setwd(base_dir)
```

```r
# Define a helper function to install packages if not already installed
install <- function(pkg) {
  if (!require(pkg, character.only = TRUE)) {
    install.packages(pkg, dependencies = TRUE, quiet = TRUE)
  }
}

# Define a helper function to load packages
load_pkg <- function(pkg) {
  message(paste("Loading package:", pkg))
  library(pkg, character.only = TRUE, quietly = TRUE)
}

# Required packages for this analysis
pkgs <- c("readxl", "vars", "urca", "tseries", "forecast", "quantmod", "rugarch",
"FinTS", "tsDyn")

# Apply install and load functions
lapply(pkgs, install)
lapply(pkgs, load_pkg)

##########
# PART A #
##########

# Downloading data for NVDA ticker from Yahoo Finance
ticker <- "NVDA"
getSymbols(ticker, src = "yahoo", from = Sys.Date() - 5*365, auto.assign = TRUE)
data <- get(ticker)

# Use Cl function from quantmod to get the adjusted close series from the data
prices <- Cl(data)

# Compute log returns
log_returns <- diff(log(prices), lag = 1)
log_returns <- na.omit(log_returns)

# Inspect head
head(log_returns)

# Plot closing prices and log returns
par(mfrow = c(2,1))
plot(prices, main = paste(ticker, "Adjusted Close Price"))
plot(log_returns, main = paste(ticker, "Daily Log Returns"), col = "blue")

# ARCH test
ArchTest(ts(log_returns), lags = 5)

# Scale returns by 100 for better convergence
scaled_returns <- log_returns * 100

# Define ARCH(1) specification
spec_arch <- ugarchspec(
```

```r
  variance.model = list(model = "sGARCH", garchOrder = c(1, 0)),
  mean.model = list(armaOrder = c(0, 0), include.mean = TRUE),
  distribution.model = "norm"
)

# Fit ARCH(1) model
fit_arch <- ugarchfit(spec = spec_arch, data = scaled_returns)
show(fit_arch)

# Define GARCH(1,1) specification
spec_garch <- ugarchspec(
  variance.model = list(model = "sGARCH", garchOrder = c(1, 1)),
  mean.model = list(armaOrder = c(0, 0), include.mean = TRUE),
  distribution.model = "norm"
)

# Fit GARCH(1,1) model
fit_garch <- ugarchfit(spec = spec_garch, data = scaled_returns)
show(fit_garch)

# Compare model AIC
cat("ARCH(1) AIC:", infocriteria(fit_arch)[1], "\n")
cat("GARCH(1,1) AIC:", infocriteria(fit_garch)[1], "\n")

# Forecast next 90 days
forecast_days <- 90
garch_forecast <- ugarchforecast(fit_garch, n.ahead = forecast_days)
forecast_vol <- sigma(garch_forecast)

# In-sample volatility and its time index
in_sample_vol <- sigma(fit_garch)
in_sample_idx <- index(in_sample_vol)

# Get last 90 days of conditional volatility from the sample
n_hist <- 90
last_hist_idx <- tail(index(in_sample_vol), n_hist)      # 90 in-sample dates
last_hist_vol <- tail(in_sample_vol, n_hist)             # 90 in-sample
volatilities

# Get the next 90 days after the last date of the sample
first_forecast_date <- last(last_hist_idx) + 1
forecast_idx <- seq.Date(from = first_forecast_date, by = "day", length.out =
forecast_days)

# Combine all dates and all volatilities
all_dates <- c(last_hist_idx, forecast_idx)
all_vol   <- c(as.numeric(last_hist_vol), as.numeric(forecast_vol))

# Plot the Conditional Volatility of the last 90 days and the forecast for the
next 90 days
plot(all_dates, all_vol, type = "l", col = "blue",
     ylab = "Volatility (%)",
     xlab = "Date",
     main = "Conditional Volatility and 90-Day Forecast")
```

```
abline(v = last(last_hist_idx), col = "red", lty = 2)
legend("topright", legend = c("In-sample + Forecast", "Forecast Start"),
       col = c("blue", "red"), lty = c(1,2))

##########
# Part B #
##########

# File Config
datasets_dir <- "datasets"
datasets_path <- file.path(base_dir, datasets_dir)
file_name <- "CMO-Historical-Data-Monthly.xlsx"
file_path <- file.path(datasets_path, file_name)

# Read Excel File and handle the Date column
data_raw <- read_excel(file_path, sheet = "Monthly Prices", skip = 6)
data_raw <- as.data.frame(data_raw)
names(data_raw) <- trimws(names(data_raw))
names(data_raw)[1] <- "Year_Month"
data_raw$Year_Month <- as.Date(paste0(
  substr(data_raw$Year_Month, 1, 4), "-",
  substr(data_raw$Year_Month, 6, 7), "-01"))

# Clean up undesirable string patterns for as.numeric conversion
clean_numeric_column <- function(vec) {
  vec <- as.character(vec)
  vec <- trimws(vec)
  vec[vec %in% c("", "-", "NA", "n.a.", "N/A", ".", "#N/A N/A")] <- NA
  suppressWarnings(as.numeric(vec))
}

# Data pre-processing per category
prepare_category_df <- function(df, cols) {
  # Subset data frame to only needed columns (assumes df includes Year_Month)
  df_subset <- df[, c("Year_Month", cols), drop=FALSE]

  # Clean columns for numeric conversion
  for (col in cols) {
    df_subset[[col]] <- as.character(df_subset[[col]])
    df_subset[[col]] <- trimws(df_subset[[col]])
    # Replace common non-numeric placeholders with NA
    df_subset[[col]][df_subset[[col]] %in% c("", "-", "NA", "n.a.", "N/A", ".",
"#N/A N/A")] <- NA
    # Then convert to numeric
    df_subset[[col]] <- suppressWarnings(as.numeric(df_subset[[col]]))
  }

  # Remove any rows with NA
  n_before <- nrow(df_subset)
  df_subset <- na.omit(df_subset)
  n_after <- nrow(df_subset)
  cat(sprintf("Rows before cleaning: %d. After omitting NA: %d. Removed: %d\n",
n_before, n_after, n_before - n_after))
```

```
  # Convert to base data.frame to allow rownames
  df_subset <- as.data.frame(df_subset)

  # Set row names as dates for convenience (optional)
  rownames(df_subset) <- as.character(df_subset$Year_Month)

  # Remove Year_Month column to keep only numeric columns
  df_subset$Year_Month <- NULL

  # Verify all columns numeric and no NAs remain
  for (col in cols) {
    if (!is.numeric(df_subset[[col]])) stop(paste("Column", col, "is not numeric
after cleaning!"))
    if (any(is.na(df_subset[[col]]))) stop(paste("Column", col, "still contains NA
after cleaning!"))
  }

  # Log transform prices
  log_df <- log(df_subset)

  # Differencing log prices
  log_diff_df <- diff(as.matrix(log_df))
  log_diff_df <- as.data.frame(log_diff_df)

  return(list(log = log_df, diff_log = log_diff_df))
}

# Augmented Dickey-Fuller test per column for stationarity of differenced log
prices
adf_test <- function(diff_df) {
  cat("ADF Test Results (on differenced log prices):\n")
  for (col in colnames(diff_df)) {
    res <- adf.test(diff_df[[col]])
    stationary <- ifelse(res$p.value < 0.05, "Stationary", "Non-stationary")
    cat(sprintf("%-15s: ADF Stat = %.3f, p-value = %.4f --> %s\n", col,
res$statistic, res$p.value, stationary))
  }
}

# VAR lag selection, fitting, impulse response plotting
run_var <- function(diff_df) {
  lag_sel <- VARselect(diff_df, lag.max = 12, type = "const")
  cat("Lag order selection (AIC, BIC, HQIC):\n")
  print(lag_sel$selection)

  # Extract lag as numeric scalar (important!)
  selected_lag <- as.numeric(lag_sel$selection["AIC(n)"])
  cat("Selected lag order by AIC:", selected_lag, "\n")

  # Fit VAR model with numeric lag
  var_model <- VAR(diff_df, p = selected_lag, type = "const")

  cat("VAR Model summary:\n")
  print("VAR Model object created. Checking its structure:")
```

```
  print(summary(var_model))

  # Impulse response function plots
  irf_res <- irf(var_model, n.ahead = 10, ortho = FALSE, boot = FALSE)
  plot(irf_res)

  return(list(model = var_model, lag = selected_lag))
}

# Johansen cointegration test wrapper and estimation
run_johansen <- function(log_df, lag_diff) {
  johansen_result <- ca.jo(log_df, type = "trace", ecdet = "const", K = lag_diff +
1)
  cat("Johansen cointegration test summary:\n")
  print(summary(johansen_result))
  return(johansen_result)
}

get_coint_rank <- function(johansen_result) {
  trace_stats <- johansen_result@teststat
  crit_vals <- johansen_result@cval[, "5pct"]
  estimated_rank <- sum(trace_stats > crit_vals)
  cat(sprintf("Estimated cointegration rank (5%% significance): %d\n",
estimated_rank))
  return(estimated_rank)
}

fit_vecm <- function(log_df, coint_rank, lag_diff) {
  vecm_model <- VECM(log_df, lag = lag_diff, r = coint_rank, estim = "ML")
  cat("VECM fit summary:\n")
  print(summary(vecm_model))
}

# High-level function for category-wise analysis
do_analysis <- function(df, category_name, cols) {
  cat("\n========================================\n")
  cat(sprintf("Analyzing %s Commodities\n", category_name))
  cat("========================================\n")

  # Check all columns present
  if (!all(cols %in% colnames(df))) {
    missing_cols <- cols[!cols %in% colnames(df)]
    stop(paste("Error: Missing columns in data frame:", paste(missing_cols,
collapse = ", ")))
  }

  prepared <- prepare_category_df(df, cols)
  log_df <- prepared$log
  diff_log_df <- prepared$diff_log

  cat("\nADF Test for Stationarity:\n")
  adf_test(diff_log_df)

  cat("\nFitting VAR Model:\n")
```

```
  var_results <- run_var(diff_log_df)
  selected_lag <- as.numeric(var_results$lag)

  cat("\nConducting Johansen Cointegration Test:\n")
  johansen_res <- run_johansen(log_df, lag_diff = selected_lag)

  rank <- get_coint_rank(johansen_res)
  if (rank > 0) {
    cat("\nFitting VECM Model:\n")
    fit_vecm(log_df, rank, selected_lag)
  } else {
    cat("\nNo significant cointegration detected; skipping VECM.\n")
  }
}

# ---- Perform Analysis on Commodity Groups and Subgroups ----

# Define your analysis groups (edit these lists for your own use case)
crude_oil_cols    <- c('CRUDE_BRENT', 'CRUDE_WTI', 'CRUDE_DUBAI')
natural_gas_cols  <- c('NGAS_US', 'NGAS_EUR', 'NGAS_JP')
coal_cols         <- c('COAL_AUS', 'COAL_SAFRICA')
metals_cols       <- c('GOLD', 'SILVER', 'COPPER', 'ALUMINUM')
wheat_cols        <- c('WHEAT_US_HRW', 'WHEAT_US_SRW')
sugar_cols        <- c('SUGAR_EU', 'SUGAR_US', 'SUGAR_WLD')

do_analysis(data_raw, "Crude Oil", crude_oil_cols)
do_analysis(data_raw, "Natural Gas", natural_gas_cols)
do_analysis(data_raw, "Coal", coal_cols)
do_analysis(data_raw, "Metals", metals_cols)
do_analysis(data_raw, "Wheat", wheat_cols)
do_analysis(data_raw, "Sugar", sugar_cols)
```